Informe de estado y plan de desarrollo

Grupo N° 4 | Catalina Giordano y Francisco Ledesma

Revisión: 1 de octubre de 2026. Rama analizada: **main**.
Commit: **708456a2252eeb1077967bce3ac80895fc107467**.

Conclusión principal

El proyecto tiene una base de backend organizada para administrar mascotas, adoptantes y solicitudes de adopción. Cuenta con separación en capas, persistencia SQLite, contratos DTO, migración inicial y pruebas automatizadas escritas. Todavía no constituye el portal completo descrito en los 18 casos de uso: faltan la interfaz web, las cuentas y permisos, el circuito de entrevista y prueba, el historial clínico y los módulos de control y seguimiento.

La diferencia más importante es funcional: **aprobar una solicitud marca inmediatamente a la mascota como Adoptada**. Según la especificación del propio repositorio, la aprobación debe habilitar una entrevista y luego un período de prueba; la adopción definitiva corresponde a CU-15.

Resultado de la revisión	Situación comprobada
Código principal	3 entidades, 3 controladores, 3 servicios y 3 repositorios concretos.
Casos de uso	18 especificados: 5 con soporte parcial y 13 sin implementación identificada.
Pruebas	73 métodos [Fact]: 37 de negocio y 36 de integración; las 73 pruebas originales aprobaron.
GitHub	Solo main; consulta pública sin Pull Requests abiertos ni cerrados.
Ramas y revisión cruzada	Pendientes en remoto. Este informe incluye una propuesta de reparto y procedimiento.

Objetivo y alcance

Contrastar archivos y reglas del repositorio con el comportamiento visible en el código; identificar faltantes; proponer cómo completar cada área y organizar ramas y Pull Requests por caso de uso. Se verificó la compilación. La evaluación funcional del repositorio original es estática; las pruebas se detallan al final.

Repositorio: github.com/PracticasProfesionalizantes-I-2026/Grupo-N-4-PortalAdopcion. La guía adjunta se utilizó como marco de trazabilidad entre interfaz, HTTP, controlador, negocio y persistencia; no como prueba de funcionalidades existentes.

1. Qué tiene actualmente

Componente	Implementación observada
Arquitectura	API → BusinessLogic → DataAccess; Shared concentra DTOs, enums y excepciones. Inyección de dependencias en API/Program.cs.
Mascotas	GET listado y por ID, POST, PUT y DELETE. Filtro por estado; validación de nombre, raza y edad de 0 a 30.
Adoptantes	CRUD de datos personales. Nombre, email, DNI, teléfono y dirección; comprobación de duplicados e índices únicos de email/DNI.
Solicitudes	Listado, consulta por ID, creación, cambio de estado y cancelación lógica mediante DELETE con motivo.
Reglas básicas	Solo mascota Disponible; una solicitud Pendiente por mascota y por adoptante. Crear reserva la mascota; rechazar/cancelar exige motivo y libera la reserva.
Persistencia	EF Core con SQLite; InitialCreate en Migrations/. Relaciones restrictivas, enums como texto y lecturas AsNoTracking.
Consistencia de escritura	Solicitud y estado de mascota se guardan juntos mediante SaveChangesAsync. Esto no resuelve por sí solo validaciones concurrentes previas.
Respuesta HTTP	CreatedAtAction devuelve 201 y referencia al recurso. Errores de servicios se mapean a 400, 404 y 409.
Herramientas y pruebas	OpenAPI y Scalar en desarrollo; inicializador con migración y datos de ejemplo; xUnit/Moq y pruebas HTTP con SQLite en memoria.
Documentación	README, AGENTS.md, especificaciones CU-01 a CU-18 y materiales de análisis en COSAS/.

Trazabilidad real de una postulación

POST /api/solicitudes con mascotaId y adoptanteId → SolicitudesController.Create → SolicitudService.CreateAsync → SolicitudRepository.CreateAsync → PortalAdopcionDbContext → SaveChangesAsync → respuesta 201. Se crea la solicitud Pendiente y la mascota pasa a Reservada.

Todavía no hay una pantalla ni un botón Guardar implementados en el repositorio. El flujo empieza en una petición HTTP. Tampoco se identifican autenticación JWT, código de confirmación, auditoría o notificaciones.

Aplicación práctica de la guía adjunta

La separación en capas y el uso de DTOs ya están presentes. Los endpoints actuales no están versionados como /api/v1, los JSON usan camelCase y los errores de negocio usan { message }. Son diferencias de contrato que deben acordarse y documentarse; snake_case, HATEOAS o CQRS no son requisitos indispensables para terminar este proyecto.

2. Estado de los 18 casos de uso

Parcial: existe código relacionado, pero faltan condiciones de la especificación. **Pendiente**: se encontró la documentación, sin implementación específica. Ningún CU se considera completo frente al alcance total documentado.

CU	Función	Estado	Cómo completar
01	Blacklist	Pendiente	Crear sanciones vigentes, motivo, duración y bloqueo de postulaciones.
02	Crear solicitud	Parcial	Agregar identidad autenticada, perfil completo, blacklist y confirmación.
03	Historial clínico inicial	Pendiente	Crear historial único por mascota con primer registro clínico.
04	Evaluar solicitud	Parcial	Cumplir 15 días, observación, autorización y auditoría; no adoptar todavía.
05	Crear proceso automático	Parcial	Integrar con CU-02; faltan confirmación y auditoría. No duplicar entidad.
06	Registrar usuario	Pendiente	Crear cuenta con contraseña protegida y rol. CRUD de adoptante no es cuenta.
07	Iniciar sesión	Pendiente	Verificar credenciales y estado; emitir token y probar 401/403.
08	Usuarios y roles	Pendiente	Gestión administrativa de roles y estados de cuenta.
09	Catálogo de mascotas	Parcial	Permisos, foto, baja lógica, estados ampliados y auditoría.
10	Evento médico	Pendiente	Registrar eventos sobre historial, con permisos veterinarios.
11	Entrevista	Pendiente	Fecha, responsable, resultado y paso al período de prueba.
12	Seguimiento de prueba	Pendiente	Entradas diarias, alertas, asignaciones y límite de tareas acordado.
13	Cancelar proceso	Parcial	Ampliar estados cancelables; responsable, auditoría y sanción en prueba.
14	Abandono automático	Pendiente	Job diario, más de 7 días sin actividad en entrevista; liberar reserva.
15	Finalizar adopción	Pendiente	Verificar período de prueba y alertas; conservar acceso al historial.
16	Reportes	Pendiente	Consultas por fechas y exportación CSV/PDF autorizada.
17	Consultar auditoría	Pendiente	Registro de eventos y consulta administrativa solo lectura.
18	Reportar incidente	Pendiente	Gravedad, detalle, asignación; urgentes visibles para administración.

CU-05 es una subfunción automática de CU-02 según la documentación actual. Mantener su trazabilidad por separado no justifica crear otro proceso, otra solicitud o un endpoint duplicado. CU-15 tampoco está implementado por el simple hecho de que hoy se use el estado Adoptada.

3. Diferencias que deben corregirse primero

1. La evaluación salta el circuito de adopción

Evidencia: SolicitudService.UpdateEstadoAsync asigna Adoptada cuando recibe Aprobada. EstadoSolicitudEnum solo tiene Pendiente, Aprobada, Rechazada y Cancelada.
Acción: ampliar estados y definir transiciones; Aprobada mantiene reserva y habilita entrevista. Adoptada se aplica al finalizar CU-15. Actualizar las pruebas que actualmente esperan adopción inmediata.

2. Falta autenticación y control de permisos

Evidencia: API/Program.cs no registra autenticación/autorización y los tres controladores no tienen [Authorize]. SolicitudCreateDTO acepta adoptanteId enviado por el cliente.
Acción: implementar cuentas y sesión, aplicar políticas por rol y derivar la identidad del adoptante de la sesión; una operación administrativa en nombre de otro usuario debe tener permiso explícito.

3. Se puede modificar el estado de la mascota por el CRUD

Evidencia: MascotaUpdateDTO expone Estado y MascotaService.UpdateAsync lo entrega al repositorio sin validar transiciones ni solicitudes activas.
Riesgo: una mascota con solicitud Pendiente podría volver a Disponible por PUT, dejando estados contradictorios.
Acción: separar la edición de datos de las transiciones de negocio y probar la invariancia mascota/solicitud.

4. La validación previa no garantiza unicidad concurrente

Evidencia: GetActivaPorMascotaAsync/TieneActivaAsync se consultan antes de guardar; los índices de solicitud son no únicos.
Riesgo por análisis estático: dos peticiones simultáneas podrían superar las verificaciones.
Acción: definir estados activos, reforzar unicidad en BD o escritura condicional/transacción con revalidación, y traducir conflictos a 409. Probar dos solicitudes simultáneas: solo una debe persistir.

5. Duplicados normalizados y enums sin validar

Evidencia: AdoptanteService consulta email/DNI antes de Trim; el repositorio compara directamente. MascotaService no usa Enum.IsDefined para especie, sexo o estado.
Acción: normalizar antes de consultar y guardar; acordar equivalencia de email; capturar violaciones de índices únicos. Rechazar valores numéricos de enum no definidos con 400.

6. Documentos y bajas no coinciden con el código

El README anuncia bruno/ pero la carpeta no está. COSAS conserva avisos de archivos perdidos aunque hoy existen los 18 CU; CU-03 ya corrige la precondición y CU-05 ya se declara subfunción. Las RN-01..04 del README no equivalen a las RN del catálogo unificado. Los DELETE de mascotas/adoptantes son físicos, mientras CU-09 pide baja administrativa.
Acción: actualizar documentos, adoptar una numeración común, crear la colección Bruno y decidir/implementar baja lógica.

4. Orden propuesto para continuar

Desarrollar por incrementos verificables. Cada entrega incluye contrato, reglas, persistencia, pruebas y documentación; agregar interfaz cuando el contrato del caso esté acordado.

Etapa	Trabajo	Condición para continuar
0. Alinear el alcance	Corregir RN y referencias desactualizadas; definir estados activos, perfil obligatorio y alcance de RN-16. Mantener CU-05 interno a CU-02.	Diagrama de estados y tabla CU → endpoint → reglas aprobados por el grupo.
1. Identidad y base	CU-06, CU-07 y CU-08; cuentas, roles, estado y relación con Adoptante. Agregar estructura de auditoría reutilizable.	Registro/login y permisos comprobados con 201/200/401/403.
2. Catálogo y postulación	Completar CU-09 y CU-01; luego CU-02/CU-05. Resolver estados editables, blacklist, perfil y reservas concurrentes.	Un usuario sancionado no postula; una mascota no admite dos procesos activos.
3. Evaluación y entrevista	CU-04 y CU-11; 15 días mínimos, observación, evaluador, entrevista y transiciones.	Aprobar no adopta; entrevista habilita prueba y registra responsable.
4. Salud y seguimiento	CU-03 y CU-10; luego CU-12 y CU-18. Historial único, eventos, tareas, seguimiento y alertas.	Una alerta impide finalizar sin validación veterinaria.
5. Cierre y abandono	CU-13, CU-14 y CU-15. Cancelación por etapa, sanción de un año, job de abandono y finalización.	Reservas liberadas correctamente; finalización conserva historial y auditoría.
6. Control y entrega	CU-17 y CU-16; consultas auditadas, reportes y exportación. Completar interfaz web por rol y documentación de arranque.	Demostración desde pantalla hasta BD, Bruno y suite de pruebas.

Entidades que harán falta

Identidad: Usuario/Rol o infraestructura equivalente, relación con Adoptante y estado de cuenta. **Control:** SancionBlacklist y EventoAuditoria. **Proceso:** Entrevista, datos de evaluación, fechas de actividad y de prueba. **Salud:** HistorialClinico y EventoMedico. **Seguimiento:** RegistroSeguimiento, asignaciones/tareas e Incidente. Son propuestas de modelado; definir campos y cardinalidades antes de migrar.

Decisiones de alcance que ya deben respetarse

La especificación de CU-18 mantiene los incidentes urgentes visibles para administración y no exige correo automático fuera del MVP. No ampliar esa obligación sin acuerdo. En CU-02, definir el mecanismo de confirmación y notificaciones para el alcance aprobado. RN-16 indica expresamente que parte del seguimiento está pendiente de confirmación.

Una migración por incremento coherente

Agregar entidad y mapeo, repositorio e interfaz, servicio, DTOs, controlador y registro en DI. Generar migración conforme AGENTS.md. Revisar el SQL y los datos existentes antes de aplicar cambios de estados o índices; no borrar la base para ocultar incompatibilidades.

5. Ramas y responsables propuestos

El README identifica a Catalina Giordano y Francisco Ledesma; la consulta pública identifica las cuentas catagiordano y franledesma04. El reparto siguiente es **una propuesta, no una asignación acordada ni una creación en GitHub**. F = Francisco (revisa Catalina); C = Catalina (revisa Francisco).

CU	Rama al comenzar su desarrollo	Autor / revisor
01	feature/cu-01-blacklist	Francisco / Catalina
02	feature/cu-02-solicitud-adopcion	Francisco / Catalina
03	feature/cu-03-historial-clinico	Catalina / Francisco
04	feature/cu-04-evaluar-solicitud	Francisco / Catalina
05	feature/cu-05-creacion-proceso	Francisco / Catalina
06	feature/cu-06-registro-usuario	Catalina / Francisco
07	feature/cu-07-iniciar-sesion	Catalina / Francisco
08	feature/cu-08-usuarios-roles	Catalina / Francisco
09	feature/cu-09-catalogo-mascotas	Francisco / Catalina
10	feature/cu-10-evento-medico	Catalina / Francisco
11	feature/cu-11-entrevista	Francisco / Catalina
12	feature/cu-12-seguimiento-prueba	Catalina / Francisco
13	feature/cu-13-cancelar-proceso	Francisco / Catalina
14	feature/cu-14-abandono-automatico	Francisco / Catalina
15	feature/cu-15-finalizar-adopcion	Francisco / Catalina
16	feature/cu-16-reportes	Catalina / Francisco
17	feature/cu-17-auditoria	Catalina / Francisco
18	feature/cu-18-incidentes	Catalina / Francisco

El reparto cubre los 18 CU de forma equilibrada en cantidad; el esfuerzo no es idéntico. Crear las ramas desde main actualizada cuando el CU pueda iniciarse, respetando las dependencias de la página anterior. No abrir 18 PR vacíos ni repartir el mismo cambio entre varios PR.

Tratamiento de CU-05

Para mantener una rama/PR por CU sin duplicar funcionalidad, CU-02 implementa la entrada y CU-05 completa o encapsula la subfunción de creación, confirmación y trazabilidad sobre el mismo modelo. CU-05 se inicia tras integrar la base de CU-02. Si toda la subfunción ya queda en CU-02, acordar con el docente un PR conjunto identificado como CU-02/CU-05.

El código actual ya está integrado en main. Los nuevos PR deben mostrar mejoras reales respecto de esa base. Crear una rama apuntando al mismo commit no genera una diferencia revisable.

6. Procedimiento de trabajo y revisión cruzada

Crear y publicar una rama de trabajo

Ejemplo: Francisco empieza por completar CU-09. Catalina puede desarrollar CU-06 en otra rama. Ejecutar desde el repositorio local:

```
git switch main
git pull --ff-only origin main
git switch -c feature/cu-09-catalogo-mascotas

# Implementar y verificar antes de publicar
dotnet build PortalAdopcion.slnx
dotnet test PortalAdopcion.slnx
git status
git add <archivos-del-caso>
git commit -m "CU-09: completar catalogo de mascotas"
git push -u origin feature/cu-09-catalogo-mascotas
```

Abrir el Pull Request en GitHub

Elegir **Compare & pull request**; base **main**, compare **feature/cu-09-catalogo-mascotas**. Título concreto: “CU-09: proteger transiciones y agregar baja administrativa”. Describir el problema, el comportamiento final, los endpoints, la migración y las pruebas realizadas. Asignar al otro integrante como Reviewer; confirmar que tiene acceso al repositorio.

Contenido mínimo de cada PR

Caso y resultado: qué acción del usuario queda resuelta.
Cambios: contrato HTTP, reglas y datos afectados.
Validación: comandos ejecutados y resultados reales; ejemplos Bruno de éxito y error.
Dependencias: otros CU que deben estar integrados.
Evidencias: pantalla o respuesta HTTP y comprobación de persistencia.
Pendientes: limitaciones reales que afectan la aceptación del caso.

Revisión y aprobación

El revisor descarga la rama y comprueba el flujo principal y alternativos contra el CU, ejecuta build/tests y revisa permisos, transiciones, migraciones y errores HTTP. Si detecta problemas, usa **Request changes**. El autor corrige en la misma rama y hace push; el PR se actualiza. El revisor verifica el último cambio y usa **Approve**. Después se integra y se elimina la rama si el grupo lo acuerda.

La revisión debe hacerla el otro integrante. Una aprobación formal sin leer ni probar el caso no cumple la revisión cruzada. Configurar main para requerir un PR y al menos una aprobación cuando el plan/permisos de GitHub lo permitan. Agregar un workflow que ejecute build y test con .NET 10 y exigir sus resultados.

No publicar archivos de ejecución

Antes de cada commit revisar git status: evitar bases SQLite locales, bin/obj y credenciales. Incluir código, pruebas, migraciones, documentación y peticiones Bruno correspondientes al caso.

7. Primeros incrementos concretos y aceptación

Francisco: completar CU-09 sobre el CRUD existente

Cambio: proteger cambios de estado para impedir contradicciones con solicitudes; agregar baja lógica y campos del catálogo acordados. Conservar historial y solicitudes. Los permisos quedan vinculados al incremento de identidad de Catalina.
Pruebas: alta válida → 201; datos/enums inválidos → 400; ID inexistente → 404; transición incompatible con proceso activo → 409; sin sesión → 401 y rol insuficiente → 403 cuando identidad esté integrada.
Entrega: DTOs, servicio, repositorio, migración si corresponde, tests, CU actualizado y colección Bruno.

Catalina: desarrollar CU-06 y luego CU-07

Cambio: separar cuenta de usuario del perfil Adoptante; almacenar contraseña mediante un mecanismo de hashing de identidad, nunca como texto. El registro público crea rol Adoptante; no permite autoconcederse Administrador. Acordar con CU-08 el alta del personal.
Pruebas: datos válidos → 201; inválidos → 400; email/DNI duplicados → 409; login válido → 200; credenciales inválidas → 401; estado de cuenta según especificación.
Entrega: modelo, contratos, endpoints, migración, configuración de autenticación y pruebas; CU-07 en su propia rama y PR después de CU-06.

Pruebas de negocio prioritarias al extender solicitudes

Escenario	Resultado exigido
Aprobar antes de 15 días	409 conforme CU-04; no cambia el estado ni adopta la mascota.
Aprobar cuando corresponde	Aprobada; mascota reservada; entrevista habilitada; decisión auditada.
Dos postulaciones simultáneas	Una sola solicitud activa persistida; la otra devuelve conflicto.
Adoptante sancionado	No puede postular; conserva acceso permitido por la especificación.
Cancelar durante prueba	Motivo y responsable; libera reserva; sanción de un año.
Entrevista sin actividad >7 días	Abandonada y reserva liberada; ejecución diaria idempotente.
Finalizar con alerta	Bloqueo hasta validación veterinaria; no adopta prematuramente.

Criterio de terminado por caso de uso

Contrato y CU coinciden; reglas comprobadas; migración incluida y compatible; pruebas ejecutadas; ejemplo reproducible en Bruno; errores y permisos coherentes; documentación actualizada; revisión cruzada del último commit. Si el caso requiere pantalla, verificar el recorrido desde la UI hasta la BD y la respuesta al usuario.

No basta con que el endpoint devuelva 200/201: la reserva, los estados, la identidad y los datos persistidos también deben ser correctos.

8. Evidencias, límites y situación de la consigna

Fuentes revisadas

Versión: main, commit 708456a2252eeb1077967bce3ac80895fc107467 (“ACTUALIZADO. FALTAN COSAS”).
Arquitectura: AGENTS.md, README.md, API/Program.cs y los cuatro proyectos .csproj.
Código funcional: API/Controllers/, BusinessLogic/Services/, DataAccess/Entities/, DataAccess/Repositories/, PortalAdopcionDbContext.cs y DbInitializer.cs.
Contratos: Shared/DTOs/, Shared/Enums/ y Shared/Exceptions/.
Especificación: los 18 archivos de CASOS DE USO/; COSAS/Modelo-de-Estados-y-Reglas-de-Negocio.md, Indice-Casos-de-Uso-y-Trazabilidad.md y Pendientes-Alta-Prioridad.md.
Pruebas y esquema: Tests/ y Migrations/.
Guía adjunta: Guía Didáctica: Trazabilidad End-to-End de un Caso de Uso (Del Botón Guardar a SQLite), especialmente el mapa de trazabilidad y la plantilla de especificación.

Qué se pudo verificar

Se descargó el repositorio y se inspeccionó su código. Se contaron **37 métodos [Fact] de servicios y 36 de integración**, todos en seis archivos de pruebas. Se verificó la ausencia de bruno/ y de workflows versionados en .github. Git ls-remote y la API pública de GitHub mostraron solo main; la consulta de Pull Requests con state=all no devolvió resultados. Las cuentas contribuyentes observadas fueron franledesma04 y catagiordano.

Ejecución y límites de verificación

Después del relevamiento se habilitó el SDK .NET 10.0.401 y el repositorio original compiló con cero errores y cero advertencias. Se ejecutaron las 73 pruebas originales: 37 de negocio y 36 de integración, todas aprobadas. El resumen incluido en el ZIP detalla también las pruebas de la copia ampliada. Los riesgos de concurrencia y normalización se deducen del código y requieren reproducción controlada. No se revisaron interfaces o trabajos que no estén en el repositorio.

Situación de las ramas y Pull Requests

No se crearon ramas ni Pull Requests remotos: no hay una conexión de GitHub con acceso de escritura disponible para esta tarea. Tampoco se realizaron asignaciones de responsables, solicitudes de revisión ni aprobaciones. La tabla de ramas y el procedimiento son una propuesta lista para aplicar. Para continuar desde esta conversación, instalar y conectar GitHub con acceso al repositorio.

Los PR deben contener cambios reales de los CU elegidos; las revisiones y aprobaciones corresponden a los integrantes. El informe no afirma que funcionalidades pendientes ya estén desarrolladas ni que la consigna de revisión cruzada se haya cumplido.

Resultado esperado de la próxima entrega

Un incremento de catálogo y otro de identidad, cada uno con su rama y PR, pruebas ejecutadas y revisión del compañero. A partir de esa base, avanzar por postulación, evaluación, entrevista, prueba y finalización; después completar control, reportes y la interfaz. Actualizar este diagnóstico con los PR integrados y los resultados de ejecución.